

An Introduction to E-Commerce

with a Focus on Machine Learning, Deep Learning, and Artificial Intelligence

Lecture 12: MLOps + DataOps for E-Commerce

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences
Mansoura University
Egypt

March 23, 2026

Session plan (90 minutes)

- 1 Why MLOps and DataOps matter in e-commerce
- 2 From experimentation to production systems
- 3 What MLOps covers
- 4 Reproducibility
- 5 Pipelines and orchestration
- 6 Managerial implications

Learning objectives

By the end of this lecture, you should be able to:

- Explain why production ML in e-commerce requires disciplined operational practices beyond model training.
- Distinguish MLOps concerns from DataOps concerns while understanding their overlap in end-to-end commerce systems.
- Analyze training-serving skew, feature consistency, monitoring, drift, and deployment risk in production ML workflows.
- Design an ML system artifact set that includes reproducibility, observability, and governance requirements.

Why MLOps and DataOps matter in e-commerce

- How is the model trained repeatedly?
- How are features computed consistently?
- How is the model deployed safely?
- How is degradation detected?
- How is the system rolled back when behavior becomes harmful?

From experimentation to production systems

- **The production gap:** missing or delayed features,; inconsistent identifiers across systems,
- **Why e-commerce is especially operational:** product assortments change,; promotions alter behavior,

What MLOps covers

- **Typical MLOps responsibilities:** versioning code, data references, and model artifacts,; training pipelines,
- **What DataOps covers:** data pipeline reliability,; data contracts and validation,

- **What should be reproducible:** source code version,; feature definitions,
- **Why this matters in commerce:** new data,; a model change,

- **Typical pipeline stages:** ingest and validate raw data,; transform features,
- **Batch and streaming coexistence:** batch pipelines for training and aggregate features,; streaming or near-real-time pipelines for serving-time context such as session behavior, current inventory, or recent fraud indicators.

Feature consistency and training-serving skew

- **How skew happens:** using different code paths offline and online,; stale or delayed online feature values,
- **Why skew is dangerous:** a fraud model may receive delayed account-age features online,; a recommender may use a stale inventory flag,
- **Feature stores as a concept:** Feature stores are often introduced to reduce this risk by standardizing feature definitions and access patterns across training and serving.

Model registry and release management

- **Why a registry matters:** which model versions exist,; which one is approved,
- **Release strategies:** shadow deployment,; canary rollout,

Monitoring production ML systems

- **Technical monitoring:** latency,; throughput,
- **Model monitoring:** score distribution shifts,; calibration changes,
- **Business monitoring:** click-through and conversion for recommenders,; fraud-dollar capture and false-positive burden,

Data drift and concept drift

- **Data drift:** seasonal traffic changes,; new product categories,
- **Concept drift:** the same browsing behavior stops indicating purchase intent after a UI redesign,; fraud patterns evolve so past indicators weaken,
- **Response to drift:** alerting,; retraining,

- **What CI for ML should validate:** unit tests for transformations,; schema and feature checks,
- **What CD for ML should consider:** compatibility with serving infrastructure,; readiness of online features,

Experiment tracking and model comparison

- **What to track:** dataset reference,; feature set version,
- **Why this matters:** Without structured experiment tracking, teams can neither compare models honestly nor explain why a deployed model was chosen.

- **Examples of governed models:** fraud and abuse scoring,; dynamic pricing,
- **Typical governance artifacts:** model cards,; risk assessment notes,

Incident response for ML systems

- **Examples of ML incidents:** a recommender starts surfacing unavailable items,; a fraud model causes a spike in false declines,
- **Preparedness:** clear runbooks,; fallback modes,

Managerial implications

- Organizations often underestimate the operational cost of ML.
- The model may be only a small fraction of the total system effort.
- Most production pain comes from weak data contracts, poor observability, inconsistent features, and unclear ownership.